INDIAN SPACE RESEARCH ORGANIZATION

ISRO Telemetry Tracking and Command Network (ISTRAC)

INTERNSHIP REPORT

On

Optimization and Sequence Selection for GNSS Application Using AI Techniques

Submitted By:	**Under the Supervision of:**
Nidhi Prabhu	Dr.Dileep Dharmappa
1JB22CS093	Scientist–Section Head NTRS1
B.E CSE	ISTRAC–ISRO

Submitted On: 27th May, 2026

SJB Institute of Technology ,Bengaluru

Acknowledgments

The Internship was undertaken at the SCC, ISRO Telemetry Tracking and Command Network, ISRO. Being part of this distinguished department provided me with invaluable exposure to the processes and technologies that ensure the success of India's space missions, and it offered a unique environment of learning and collaboration.

I am deeply thankful to Dr. A. K. Anil Kumar, Director of ISTRAC, for granting this internship opportunity and for his leadership in fostering programs that encourage learning and innovation.

My appreciation extends to Mr. B. Sankar Madaswamy, Manager, HRD, ISTRAC, ISRO, for facilitating this incredible internship opportunity with ISRO and providing the necessary resources to ensure the smooth execution of the project.

I extend my gratitude to Dr.Dileep Dharmappa, Scientist/Engineer – Section Head NTRS1 ISTRAC, ISRO for his guidance, support, and the generous time he dedicated to the project. I am sincerely grateful for the opportunity to learn from him.

Lastly, I extend my gratitude to the entire team at ISTRAC for their warm welcome and cooperation during my time at the organization and for their constant support and encouragement throughout this journey.

Certificate

This is to certify that the internship titled - **'Optimization and Sequence Selection for GNSS Using AI Techniques'** has been successfully carried out by **Nidhi Prabhu** (1JB22CS093), a bonafide student of **SJB Institute of Technology, Bengaluru**, enrolled in the Bachelor of Engineering in Computer Science Engineering with specialisation in Artificial Intelligence program for the academic period 2025–2026.

The internship was undertaken at the **Indian Space Research Organization – ISRO Telemetry, Tracking and Command Network (ISTRAC), Bengaluru**, during the period **22nd January 2026 to 27th May 2026**, in partial fulfilment of the requirements of her academic program. The intern- ship was specifically carried out under the guidance of **Dr.Dileep Dharmappa,Scientist/Engineer Section Head,NTRS1 ISRO–ISTRAC**. The project involved the design, development, and evaluation of deep learning models for spacecraft attitude prediction using simulated onboard telemetry data such as position vectors, velocity vectors, and magnetometer readings under body rate conditions.

Dr.Dileep Dharmappa,
Scientist Section Head,
NTRS1 ISRO–ISTRAC

Declaration

I, Nidhi Prabhu(1JB22CS093), a student of Computer Science Engineering of SJB Institute of Technology, hereby declare that the internship titled - 'Optimization and Sequence Selection for GNSS Application using AI Techniques' has been successfully completed by me at the Indian Space Research Organization – ISRO Telemetry, Tracking and Command Network (ISTRAC), Bengaluru, under the guidance of Dr.Dileep Dharmappa, Scientist– Section Head,NTRS1 ISRO–ISTRAC.

This internship was carried out during the period 22nd January, 2026 to 27th May, 2026 during the duration of my Bachelor of Engineering program.

Date: 27th May, 2026
Place: Bangalore

Nidhi Prabhu
1JB22CS093

Abstract

This report presents a detailed study of important mathematical and communication concepts used in modern digital systems, with special focus on the Indian Space Research Organisation (ISRO), ISTRAC, NavIC, Galois Fields, Linear Feedback Shift Registers (LFSRs), Z4 Rings, primitive polynomials, and correlation techniques. The study explains how these technologies and mathematical structures support reliable communication, navigation, encryption, and sequence generation in modern engineering applications. ISTRAC plays a critical role in India's space missions by providing telemetry, tracking, and command support for satellites and spacecraft. NavIC demonstrates India's technological self-reliance through an independent regional navigation system designed for accurate positioning and timing services.

The report also explores the theoretical foundations of finite algebraic structures such as Galois Fields and Z4 Rings, which are widely used in coding theory, cryptography, and digital communication systems. Special emphasis is given to LFSRs and primitive polynomials because of their ability to generate maximum-length pseudorandom sequences with good statistical properties. Concepts such as autocorrelation and cross-correlation are studied to analyze sequence similarity and communication efficiency.

In addition to theoretical discussion, Python-based implementations are presented for sequence generation, balance analysis, correlation computation, and optimal sequence filtering using graph-theoretic approaches. The project demonstrates how mathematical theory can be transformed into practical computational models for communication engineering applications. Overall, the study highlights the strong relationship between abstract mathematics, digital communication, and real-world technological systems while emphasizing India's progress in space and navigation technologies.

Introduction

ISTRAC is often called the “silent backbone” of India’s space missions. While rockets, astronauts, and satellites usually receive public attention, organizations like ISTRAC work quietly behind the scenes to make every mission possible. Without its support, India’s satellites and space probes would not be able to communicate with Earth, making it impossible to control or monitor them once they leave the ground.

ISTRAC stands for the ISRO Telemetry, Tracking and Command Network. It is a specialized division of the Indian Space Research Organisation, headquartered in Bengaluru. Its main responsibility is to stay connected with spacecraft during every stage of a mission. Whether it is a satellite orbiting Earth, a spacecraft heading toward Mars, or a lunar lander touching down on the Moon, ISTRAC acts like a communication bridge between scientists on Earth and machines in space.

To understand ISTRAC better, imagine a spacecraft as a traveler on a very long journey. Just as travelers need maps, communication, and guidance, spacecraft also need constant support. ISTRAC provides this support through telemetry, tracking, and command systems. Telemetry means receiving data from spacecraft, such as temperature, speed, fuel levels, and system health. Tracking involves observing the exact location and movement of the spacecraft. Command means sending instructions from Earth, such as changing direction, adjusting orbit, or activating instruments. These three activities together ensure that missions continue smoothly and safely.

The importance of ISTRAC grew as India’s space ambitions expanded. In the early days of India’s space program, only a few satellites were launched, and operations were relatively simple. But as ISRO began launching more advanced missions, the need for a dedicated communication and tracking network became clear. Over time, ISTRAC developed into a sophisticated system with ground stations and mission control facilities spread across India and other parts of the world.

One of the most exciting moments for ISTRAC came during Chandrayaan-1, India’s first mission to the Moon. Launched in 2008, the spacecraft traveled hundreds of thousands of kilometers away from Earth. During this journey, ISTRAC engineers constantly monitored its condition and ensured communication remained stable. The mission became historic when it helped confirm the presence of water molecules on the Moon’s surface. Although scientists and

astronauts often receive credit for such discoveries, teams at ISTRAC were equally important because they kept the spacecraft connected and functioning.

Another unforgettable achievement was the Mars Orbiter Mission in 2013. Reaching Mars is considered one of the most difficult challenges in space exploration because of the enormous distance involved. Signals traveling between Earth and Mars can take several minutes to arrive, and even small errors can lead to mission failure. ISTRAC handled the delicate task of tracking the spacecraft throughout its long journey. When India successfully entered Mars orbit on its first attempt, it was a proud moment not only for ISRO but also for every engineer working behind the scenes at ISTRAC.

More recently, ISTRAC played a major role in Chandrayaan-3, which achieved a successful soft landing near the Moon's south pole in 2023. During the tense landing sequence, scientists at mission control closely monitored incoming data from the spacecraft. Every signal received by ISTRAC brought relief and excitement. When the Vikram lander touched down safely, celebrations erupted across the country. For many people, it was a reminder that space missions are not only about machines and technology but also about teamwork, dedication, and human effort.

ISTRAC's work does not end after a mission is launched. It continues to support satellites throughout their operational life. India uses satellites for weather forecasting, communication, disaster management, television broadcasting, navigation, and environmental monitoring. These satellites help farmers predict rainfall, assist fishermen in navigation, improve internet and communication systems, and support rescue operations during natural disasters. ISTRAC ensures these satellites remain healthy and operational by continuously monitoring them and sending necessary commands when required.

The organization also operates powerful antennas and communication systems capable of receiving signals from spacecraft located millions of kilometers away. Some of these signals are extremely weak by the time they reach Earth. Engineers at ISTRAC must therefore rely on advanced technology and precision to interpret the data correctly. This requires not only scientific expertise but also patience and concentration.

One interesting aspect of ISTRAC is the human side of its operations. During major missions, scientists and engineers often work for long hours without rest. The pressure can be enormous because even a small mistake may affect an entire mission. Yet the teams continue working with determination and calmness. Images from mission control rooms often show scientists

anxiously watching screens, waiting for confirmation signals from spacecraft. These moments reveal the emotional side of space exploration — the tension, hope, relief, and joy experienced by the people behind the missions.

ISTRAC also represents India's growing confidence in space technology. Over the years, it has evolved from supporting basic satellite launches to handling complex interplanetary missions. Its achievements have earned international respect, and it frequently collaborates with global space agencies for tracking and mission support. Such partnerships highlight India's increasing importance in the international space community.

In the coming years, ISTRAC's responsibilities are expected to grow even further. India's planned Gaganyaan mission, which aims to send Indian astronauts into space, will require highly reliable communication and tracking systems. Human spaceflight demands even greater precision because astronauts' lives depend on flawless coordination. ISTRAC will therefore play a critical role in ensuring the safety and success of these future missions.

In conclusion, ISTRAC is much more than a technical network. It is the heartbeat of India's space missions, quietly supporting every satellite and spacecraft launched by ISRO. While rockets may capture public imagination, organizations like ISTRAC remind us that successful space exploration depends on teamwork, discipline, and constant communication. Its contributions have helped India achieve milestones that once seemed impossible, from reaching the Moon to orbiting Mars. As India continues its journey into deeper space, ISTRAC will remain one of the strongest pillars supporting the nation's dreams among the stars.

NavIC

ISRO's NavIC is one of India's most meaningful achievements in space technology because it connects directly with everyday life. Most people use navigation systems almost daily without thinking much about them. Whether someone is booking a cab, searching for directions, tracking a food delivery, or checking the fastest route home, navigation technology quietly works in the background. NavIC was created to ensure that India could provide these services independently, accurately, and securely through its own satellite network.

NavIC stands for Navigation with Indian Constellation. It is India's own regional navigation system developed by the Indian Space Research Organisation. In simple terms, NavIC works like a guide in the sky. A group of satellites orbiting Earth constantly sends signals to receivers on the ground, helping users identify their exact location and direction. It is often compared to the American GPS because both systems perform similar functions, but NavIC was specially designed to serve India and nearby regions with high accuracy.

The idea behind NavIC came from an important realization. For many years, India depended mostly on foreign navigation systems such as GPS. While these systems were useful, depending completely on another country for such an essential service was risky. During emergencies, military conflicts, or natural disasters, access to navigation information becomes extremely important. India understood that it needed its own reliable system that would always remain under national control. This thought eventually led to the development of NavIC.

Building a navigation system is not easy. It requires years of scientific research, engineering, and teamwork. Scientists had to design satellites capable of sending precise timing signals from space. Ground stations had to be established to monitor the satellites continuously. Engineers also needed to create highly accurate atomic clocks because even a tiny timing error could affect navigation accuracy. Despite these challenges, Indian scientists successfully built NavIC step by step, proving the country's growing technological strength.

The first NavIC satellite was launched in 2013, and additional satellites followed over the next few years. Together, they formed a constellation that could provide uninterrupted navigation services across India and surrounding regions. Unlike GPS, which covers the entire world, NavIC mainly focuses on India and areas extending about 1,500 kilometers beyond its borders. This regional focus allows it to provide very accurate location information within its service area.

One of the reasons NavIC is special is because it is not just about technology; it is about people. Its benefits can be seen in many parts of daily life. Imagine a fisherman going far into the sea early in the morning. In the past, poor weather or navigation mistakes could put lives at risk. With NavIC-enabled devices, fishermen can track their routes more safely and avoid accidentally crossing international maritime borders. During cyclones or storms, warning messages can also be sent quickly, giving fishermen enough time to return safely. In this way, NavIC becomes more than a machine-based system — it becomes a tool that protects lives.

Drivers and travelers also benefit from NavIC. Anyone who has experienced getting lost in traffic understands how useful accurate navigation can be. NavIC helps provide reliable directions, reduces travel confusion, and improves transportation systems. Emergency services can use accurate location data to reach accident sites faster, which can sometimes save precious minutes during medical emergencies.

In farming communities, NavIC has opened new possibilities. Agriculture today increasingly depends on technology. Farmers can use satellite-based navigation for precision farming, where water, fertilizers, and pesticides are used more efficiently. This not only improves crop production but also reduces waste and environmental damage. For a country like India, where agriculture supports millions of livelihoods, such technological support can make a significant difference.

NavIC also becomes extremely important during natural disasters. India frequently experiences floods, cyclones, earthquakes, and landslides. During such situations, rescue teams need accurate maps and location data to reach affected areas quickly. Navigation systems help coordinate relief operations, transport supplies, and identify safe routes. When communication systems are under pressure during emergencies, reliable satellite-based services become even more valuable.

Beyond civilian use, NavIC has an important role in national security. Modern defense systems rely heavily on navigation and timing technology. Aircraft, ships, drones, and missiles all need precise positioning data. If a country depends entirely on foreign systems during conflict situations, there is always uncertainty about signal availability. By developing NavIC, India ensured that its defense forces would always have access to independent navigation support when needed.

Another inspiring aspect of NavIC is what it says about India’s scientific journey. Developing such a system placed India among a small group of countries with independent satellite

navigation capabilities. It showed that Indian scientists and engineers could handle highly complex technologies that only a few nations in the world possess. The success of NavIC became a source of national pride because it represented self-reliance and innovation.

Today, NavIC is gradually becoming more visible in everyday technology. Several smartphones and electronic devices now support NavIC signals. This means ordinary users can directly benefit from India's own navigation system while using maps, travel apps, and location-based services. The government has also encouraged industries and transport systems to adopt NavIC technology more widely.

Of course, the journey has not been completely smooth. Some satellites experienced technical problems, especially with atomic clocks that are essential for maintaining timing precision. Replacing and maintaining satellites requires constant investment and effort. Since GPS has dominated global navigation for decades, convincing manufacturers and companies to adopt NavIC also takes time. However, Indian scientists continue improving the system with newer satellites and upgraded technology.

What makes NavIC truly meaningful is its quiet presence in everyday life. Most people using navigation services may not even realize which satellites are helping them. Yet behind every accurate location, safe fishing trip, faster emergency response, or guided journey, there is a network of scientists, engineers, and satellites working continuously. NavIC represents years of dedication by thousands of people who believed India could create world-class technology on its own.

NavIC also inspires future generations. Students interested in science and technology often see projects like NavIC as proof that Indian innovation can compete globally. It encourages young minds to dream bigger, whether in engineering, space science, artificial intelligence, or communication technology. In many ways, NavIC is not just about satellites in space; it is about building confidence on the ground.

Looking ahead, NavIC is expected to play an even larger role in India's digital future. As smart transportation systems, autonomous vehicles, drone delivery services, and advanced communication networks grow, accurate navigation will become even more important. NavIC has the potential to support these developments while reducing dependence on foreign systems.

In conclusion, NavIC is much more than a navigation project. It is a story of India's determination to become technologically independent and socially connected through space

science. Developed by the Indian Space Research Organisation, NavIC supports daily life in ways many people may never notice — guiding travelers, helping farmers, protecting fishermen, supporting rescue teams, and strengthening national security. Its success reflects not only scientific achievement but also the human effort, vision, and persistence behind India's growing space journey.

Galois Field

A Galois Field, also known as a finite field, is one of the most important concepts in modern mathematics, computer science, and communication engineering. It is named after the French mathematician Évariste Galois, who developed the foundations of group theory and finite fields at a very young age. Despite his short life, his ideas became extremely influential in coding theory, cryptography, digital communication, and error correction systems.

A field in mathematics is a set of numbers where operations like addition, subtraction, multiplication, and division can be performed while still remaining inside the same set. A Galois Field differs from ordinary number systems because it contains only a finite number of elements. This is why it is often written as GF(q), where "GF" stands for Galois Field and q represents the number of elements in the field.

The simplest example is GF(2), which contains only two elements: 0 and 1. Arithmetic in GF(2) follows modulo-2 rules. This means:

- $1 + 1 = 0$
- $1 + 0 = 1$
- $0 + 0 = 0$

This binary structure makes Galois Fields extremely useful in digital electronics and computer systems because computers naturally operate using binary digits.

More advanced fields are represented as GF((p^n)), where p is a prime number and n is a positive integer. For example, GF(2^3) contains eight elements. Such fields are created using irreducible or primitive polynomials that define how arithmetic operations behave within the field.

One major application of Galois Fields is in error correction coding. Whenever data is transmitted through communication channels such as satellites, mobile networks, or storage devices, errors may occur because of noise or interference. Coding techniques based on finite fields help detect and correct these errors automatically. Technologies like QR codes, CDs, DVDs, and deep-space communication systems rely heavily on Galois Field arithmetic.

Cryptography is another important area where finite fields are used. Modern encryption algorithms, including the Advanced Encryption Standard, use operations in finite fields to

secure digital information. These mathematical structures provide both efficiency and security, making them ideal for protecting sensitive data.

Galois Fields are also important in digital signal processing, computer graphics, and wireless communication systems. In spread-spectrum communication and pseudorandom sequence generation, finite field mathematics helps produce predictable yet highly complex patterns.

One interesting feature of finite fields is that every nonzero element has a multiplicative inverse. This means division is always possible except by zero. Such properties make finite fields mathematically complete and reliable for algebraic operations.

In engineering education, Galois Fields are often introduced while studying coding theory, cryptography, and sequence generation. Although the subject may initially appear abstract, it has practical importance in many technologies people use daily. Every time someone scans a QR code, stores files on a hard drive, or sends information through a satellite link, finite field mathematics is quietly working behind the scenes.

In conclusion, Galois Fields are powerful mathematical systems with finite elements that support structured arithmetic operations. Their role in modern technology is enormous, especially in communication, data security, and error correction. From binary systems to advanced encryption methods, finite fields continue to shape the digital world in fundamental ways.

Linear Feedback Shift Register

A Linear Feedback Shift Register, commonly called an LFSR, is a digital circuit used to generate sequences of binary numbers. It is widely used in communication systems, cryptography, error detection, and pseudorandom number generation. Although the structure of an LFSR is relatively simple, it can produce highly complex and useful output sequences.

An LFSR consists mainly of shift registers and feedback connections. A shift register is a group of flip-flops arranged in sequence, where binary data shifts from one position to another during each clock cycle. The “linear feedback” part refers to the method used to generate new bits. Certain bits from the register are combined using XOR (exclusive OR) operations and then fed back into the register input.

For example, consider a 4-bit LFSR. At every clock pulse, all bits shift one position to the right or left, and a new bit enters the register based on the XOR of selected previous bits. This process repeats continuously, producing a sequence of binary outputs.

One of the most important properties of an LFSR is its ability to generate pseudorandom sequences. These sequences appear random even though they are generated deterministically. If the feedback connections are chosen correctly using primitive polynomials, the LFSR can produce a maximum-length sequence, also called an m-sequence. For an n-bit register, the maximum sequence length is:

2^n - 1

This means a 4-bit LFSR can generate 15 unique states before repeating.

LFSRs are widely used in communication systems because pseudorandom sequences have excellent correlation properties. In spread-spectrum communication, such as CDMA systems, these sequences help separate users sharing the same communication channel. They are also used in satellite communication and radar systems.

Another major application of LFSRs is cryptography. Stream ciphers often use LFSRs to generate keystream sequences for encryption. Although simple LFSRs alone are not secure enough for modern cryptographic standards, combinations of multiple LFSRs can create stronger encryption systems.

LFSRs are also important in error detection and testing. In digital circuits, cyclic redundancy check (CRC) systems use structures similar to LFSRs to detect transmission errors. Hardware testing systems also use LFSRs for generating test patterns efficiently.

One reason LFSRs are popular is their hardware simplicity. They require very few logic gates and can operate at very high speeds. This makes them ideal for embedded systems and real-time applications where efficiency matters.

Despite their advantages, LFSRs also have limitations. Since they are deterministic, their sequences can eventually be predicted if the structure and initial state are known. Therefore, additional techniques are often combined with LFSRs in secure communication systems.

The design of an LFSR depends heavily on primitive polynomials defined over finite fields such as GF(2). These polynomials determine the feedback taps and directly affect the sequence length and randomness properties.

In summary, a Linear Feedback Shift Register is a simple but powerful digital sequence generator. Its ability to create long pseudorandom sequences using minimal hardware has made it valuable in communication engineering, cryptography, error detection, and digital electronics. Even though the concept is mathematically elegant, its real-world applications make it one of the most practical tools in modern digital systems.

Z4 Ring

The Z4 ring, written mathematically as $\mathbb{Z}_4$ or Z4, is a mathematical structure consisting of four elements:

{0,1,2,3}

The operations in Z4 are performed using modulo-4 arithmetic. This means numbers "wrap around" after reaching 4. For example:

- $3 + 2 = 1 \bmod 4$
- $2 \times 3 = 2 \bmod 4$

Unlike finite fields such as GF(2), Z4 is called a ring because not every nonzero element has a multiplicative inverse. In Z4, only 1 and 3 have inverses, while 2 does not. Because of this property, Z4 is not considered a field.

Z4 is important in algebra, coding theory, and digital communication. Researchers discovered that some error-correcting codes become more efficient when designed over rings like Z4 instead of binary fields. A famous example is the construction of certain nonlinear binary codes using Z4 structures.

One interesting aspect of Z4 is that it contains "zero divisors." This means two nonzero elements can multiply to produce zero. For example:

2 \times 2 = 0 \pmod{4}

This property does not occur in fields and gives Z4 unique algebraic behavior.

In communication engineering, Z4-based coding techniques are studied for improving data transmission reliability. The structure also appears in signal processing and algebraic research.

Although Z4 is mathematically simpler than many advanced algebraic systems, it provides valuable insight into modular arithmetic and ring theory. Its applications in coding and digital systems make it an important topic in modern mathematics and engineering.

Primitive Polynomial

A primitive polynomial is a special type of polynomial used in finite field mathematics, coding theory, cryptography, and digital sequence generation. Primitive polynomials are extremely important because they help generate maximum-length sequences and construct finite fields such as GF((2^n)).

To understand primitive polynomials, it is useful first to understand polynomials over finite fields. In binary systems, coefficients are usually limited to 0 and 1. For example:

x^3 + x + 1 is a polynomial defined over GF(2).

A primitive polynomial is a polynomial that is both irreducible and capable of generating all nonzero elements of a finite field through repeated multiplication. An irreducible polynomial cannot be factored into smaller polynomials within the same field, similar to how prime numbers cannot be factored into smaller integers.

Primitive polynomials are essential in constructing extension fields like GF(2^3), GF(2^4), and higher-order fields. These fields are widely used in digital communication systems and cryptographic algorithms.

One of the most common applications of primitive polynomials is in Linear Feedback Shift Registers (LFSRs). The feedback taps of an LFSR are chosen according to a primitive polynomial. When this is done correctly, the LFSR produces a maximum-length pseudorandom sequence.

For an n-bit LFSR, the maximum sequence length generated using a primitive polynomial is:

2^n - 1

This property is extremely useful in spread-spectrum communication, satellite systems, and secure data transmission.

Primitive polynomials are also important in cryptography. Many encryption algorithms rely on finite field arithmetic, where primitive elements generated from primitive polynomials provide strong mathematical structures for secure operations. Stream ciphers frequently use these polynomials for sequence generation.

In error correction coding, primitive polynomials help design codes such as BCH and Reed–Solomon codes. These codes are widely used in CDs, DVDs, QR codes, deep-space

communication, and mobile networks because they can detect and correct transmission errors efficiently.

An example of a primitive polynomial over GF(2) is:

$x^4 + x + 1$

This polynomial can generate all nonzero elements in $GF(2^4)$. If used in a 4-bit LFSR, it produces a sequence of length 15 before repeating.

Primitive polynomials are carefully selected because not every irreducible polynomial is primitive. A polynomial may be irreducible but still fail to generate maximum-length sequences. Therefore, mathematical testing is required to identify primitive polynomials.

Another important feature of primitive polynomials is their connection with cyclic structures. In finite fields, nonzero elements form cyclic multiplicative groups, and primitive polynomials help identify generators of these groups.

Although the mathematics behind primitive polynomials can become highly abstract, their practical importance is enormous. Modern digital systems rely heavily on them for secure communication, efficient coding, and reliable data transmission. They form the mathematical foundation for many technologies people use every day without realizing it.

In conclusion, primitive polynomials are specialized algebraic tools used to generate finite fields and maximum-length sequences. Their applications extend across cryptography, coding theory, communication systems, and digital electronics. By enabling efficient sequence generation and reliable error correction, primitive polynomials continue to play a vital role in modern information technology.

Autocorrelation and Cross-Correlation

Autocorrelation and cross-correlation are important mathematical techniques used in signal processing, communication engineering, statistics, radar systems, image processing, and digital communication. Both methods are used to measure similarity between signals, but they differ in the type of comparison they perform. These concepts help engineers analyze patterns, detect signals, remove noise, and improve communication reliability.

Autocorrelation measures the similarity of a signal with a delayed version of itself. In simple terms, it checks how closely a signal matches itself after being shifted in time. If a signal strongly resembles its shifted version at certain intervals, the autocorrelation value becomes high. This technique is especially useful for identifying repeating patterns or periodicity in signals.

Mathematically, the autocorrelation of a discrete signal is represented as:

R_{xx}(k)=\sum x(n)x(n-k)

where:

- (x(n)) is the original signal,
- (k) is the time shift or lag,
- (R_{xx}(k)) is the autocorrelation function.

One important property of autocorrelation is that its maximum value occurs when the shift is zero because the signal perfectly matches itself. As the shift increases, similarity may decrease depending on the nature of the signal.

Autocorrelation is widely used in communication systems and pseudorandom sequence analysis. In spread-spectrum communication systems such as CDMA, ideal pseudorandom sequences have a sharp autocorrelation peak at zero shift and low values elsewhere. This property helps receivers detect the correct signal even in the presence of noise and interference.

Another important application is radar and sonar systems. Reflected signals are correlated with transmitted signals to determine the presence, distance, and speed of objects. In speech processing and music analysis, autocorrelation helps identify pitch and periodic sound patterns.

Cross-correlation, on the other hand, measures the similarity between two different signals. Instead of comparing a signal with itself, it compares one signal with another shifted version.

This helps determine whether two signals are related and identifies the amount of delay between them.

The mathematical expression for cross-correlation is:

R_{xy}(k)=\sum x(n)y(n-k)

where:

- (x(n)) and (y(n)) are two different signals,
- (k) represents the shift,
- (R_{xy}(k)) is the cross-correlation value.

If two signals are highly similar, the cross-correlation function produces a large value at the shift where they align best. If the signals are unrelated, the values remain low.

Cross-correlation is extremely useful in communication engineering. Receivers use cross-correlation to identify incoming signals and synchronize communication systems. In GPS and satellite communication, receivers correlate received signals with known reference sequences to determine timing and position accurately.

Image processing also uses cross-correlation for pattern recognition and template matching. For example, facial recognition systems compare image patterns using correlation techniques. In medical signal analysis, cross-correlation helps compare biological signals such as ECG or EEG data.

One key difference between autocorrelation and cross-correlation is that autocorrelation always compares a signal with itself, while cross-correlation compares two separate signals. Autocorrelation mainly identifies periodicity and self-similarity, whereas cross-correlation identifies relationships between different signals.

In conclusion, autocorrelation and cross-correlation are essential tools in signal analysis and communication engineering. They help detect patterns, measure signal similarity, synchronize systems, and improve data reliability. From wireless communication and radar systems to image processing and biomedical applications, these techniques play a major role in modern technology and digital systems.

The selection of an optimal initial Seed

The selection of an optimal initial seed is one of the most important considerations in the operation of a Linear Feedback Shift Register (LFSR). An LFSR is a sequential digital circuit used to generate binary sequences with properties similar to random signals. It is widely applied in communication systems, cryptography, coding theory, digital testing, and spread-spectrum technologies. While much attention is often given to the feedback polynomial used in the LFSR, the choice of the initial seed is equally significant because it strongly influences the effectiveness and reliability of sequence generation.

The initial seed refers to the starting state loaded into the shift register before the sequence generation process begins. Since an LFSR operates deterministically, every future output sequence depends entirely on this initial state and the feedback structure. A carefully selected seed ensures that the register cycles through the maximum possible number of states, producing sequences with desirable statistical and correlation properties. Poor seed selection, on the other hand, may reduce sequence quality or even prevent proper operation.

The concept of the unit group becomes relevant when analyzing LFSRs using algebraic structures such as finite fields and finite rings. In abstract algebra, the unit group consists of all elements that possess multiplicative inverses within the given structure. For LFSRs defined over finite fields, all nonzero states belong to the unit group because every nonzero element has an inverse. These invertible states are considered valid and useful for sequence generation.

The importance of selecting the seed from the unit group arises from the operational behavior of the LFSR. States outside the unit group may lead to undesirable conditions such as repetitive short cycles or lock-up states. The most critical invalid state is the zero state, which causes the LFSR to remain permanently inactive because no transitions occur once all register bits become zero. Therefore, excluding such states is necessary for maintaining continuous and meaningful sequence generation.

An optimal seed ensures that the LFSR traverses the complete state space allowed by the feedback polynomial. When primitive polynomials are used, the register can produce maximum-length sequences that cover all nonzero states before repetition occurs. These sequences exhibit good randomness properties, balanced distributions of zeros and ones, and favorable autocorrelation characteristics. Such properties are essential in communication engineering and secure digital systems.

In spread-spectrum communication systems, different initial seeds allow multiple users to generate distinct pseudorandom sequences while using the same polynomial structure. Proper seed selection minimizes interference and improves synchronization between transmitter and receiver systems. Since many communication systems rely on accurate timing and sequence alignment, the seed directly affects overall system performance.

In cryptographic applications, seed selection becomes even more critical because the predictability of the generated sequence depends heavily on the initial state. Since LFSRs are deterministic systems, knowledge of the seed can allow an observer to reconstruct the entire output sequence. Therefore, secure systems require seeds that are difficult to predict or derive. Randomized or carefully controlled seed initialization improves resistance against attacks and unauthorized sequence recovery.

The selection of optimal seeds is also important in digital testing and fault detection systems. In hardware testing applications, LFSRs generate test patterns used to evaluate circuit reliability. Carefully chosen seeds help ensure wide coverage of possible states and improve fault detection efficiency.

Another important consideration is sequence correlation. Different seeds may generate shifted versions of the same sequence. In systems involving multiple simultaneous users, selecting seeds with suitable correlation properties helps reduce mutual interference and improve signal clarity.

In conclusion, choosing an optimal initial seed from the unit group is fundamental to the efficient operation of an LFSR. Proper seed selection ensures maximum-length sequence generation, prevents invalid states, improves randomness, enhances synchronization, and strengthens system security. Although the feedback polynomial defines the mathematical structure of the register, the initial seed determines how effectively that structure performs in practical engineering applications.

Codes

Step 1:

```
import itertools
import os
import math
# --- Correct Balance Function (Your Formula) ---
def get_balance_info(bit_str):
    counts = {str(i): bit_str.count(str(i)) for i in range(4)}

    a = counts['0'] - counts['2']
    b = counts['1'] - counts['3']

    magnitude = math.sqrt(a * a + b * b)

    if magnitude == 0:
        status = "Perfectly Balanced"
    else:
        status = "Unbalanced"

    return f"Counts: {counts} | Value: ({a}) + i({b}) | Magnitude: {magnitude} | Status: {status}"
```

```
def read_input(filename):
    if not os.path.exists(filename):
        print(f"Error: File not found at {filename}")
        return None, None, None
    try:
        with open(filename, 'r') as file:
            content = [line.strip() for line in file if line.strip()]
        if len(content) < 3:
            return None, None, None

        degree = int(content[0])

        raw_coeffs = content[1].replace(',', ' ').split()
        if len(raw_coeffs) == 1 and len(raw_coeffs[0]) > 1:
            full_coeffs = [int(d) % 4 for d in raw_coeffs[0]]
        else:
            full_coeffs = [int(float(c)) % 4 for c in raw_coeffs]

        steps = int(content[2])
        return degree, full_coeffs, steps

    except Exception as e:
```

```
        print(f"Error parsing file: {e}")
        return None, None, None



def is_unit(state):
    return any(x & 1 for x in state)



def next_state(state, coeffs):
    total = 0
    for c, s in zip(coeffs, state):
        total += c * s
    new_digit = (-total) & 3
    return (new_digit,) + state[:-1]



def get_orbit(start, coeffs, max_steps, global_seen):
    visited_local = {}
    path = []
    bits = []
    current = start

    for step in range(max_steps):
        if current in global_seen:
```

```
            return None, None, path

        if current in visited_local:
            start_idx = visited_local[current]
            cycle = path[start_idx:]
            cycle_bits = bits[start_idx:]
            return cycle, "".join(map(str, cycle_bits)), path

        visited_local[current] = step
        path.append(current)
        bits.append(current[-1])
        current = next_state(current, coeffs)

    return None, None, path


def main():
    # --- Configuration ---
    input_path = r"D:\Dileep\NIDHI_V02\24input.txt"
    output_folder = r"D:\Dileep\NIDHI_V02"

    degree, full_coeffs, steps = read_input(input_path)
    if degree is None:
        return
```

```
analysis_path = os.path.join(output_folder, f"degree{degree}_analysis.txt")
sequences_only_path = os.path.join(output_folder, f"degree{degree}_sequences_only.txt")

rhs_coeffs = full_coeffs[1:]
rhs_coeffs += [0] * (degree - len(rhs_coeffs))

global_seen = set()
seed_count = 0
analysis_lines = []
pure_sequences = []

print(f"\n--- Processing Degree {degree} ---")
total_states = 4 ** degree
checked = 0

for state in itertools.product(range(4), repeat=degree):
    checked += 1
    if checked % 50000 == 0:
        print(f"Checked {checked}/{total_states} states...")

    if state in global_seen or not is_unit(state):
        global_seen.add(state)
        continue
```

```
    cycle, bit_str, path = get_orbit(state, rhs_coeffs, steps, global_seen)

    for s in path:
        global_seen.add(s)

    if cycle:
        seed_count += 1
        primary_seed = "".join(map(str, cycle[0]))
        period = len(cycle)

        # --- NEW BALANCE ANALYSIS ---
        balance_info = get_balance_info(bit_str)

        analysis_lines.append(f"Orbit #{seed_count} | Seed: {primary_seed} | Period: {period}")
        analysis_lines.append(f"Balance Info: {balance_info}")
        analysis_lines.append(f"Seq Snippet: {bit_str[:100]}...")
        analysis_lines.append("-" * 40)

        pure_sequences.append(bit_str)

        if seed_count % 5 == 0:
            print(f"Found {seed_count} cycles so far...")
```

```
    try:
        with open(analysis_path, 'w') as f:
            f.write(f"Results for Degree {degree}\nTotal Cycles: {seed_count}\n\n")
            f.write("\n".join(analysis_lines))
        print(f"✅ Saved Analysis: {analysis_path}")

        with open(sequences_only_path, 'w') as f:
            f.write("\n".join(pure_sequences))
        print(f"✅ Saved Pure Sequences: {sequences_only_path}")

    except Exception as e:
        print(f"Error saving files: {e}")



if __name__ == "__main__":
    main()
```

Step 2

new py

```
import itertools
import math

# ------------------ File Paths ------------------
input_file = r"D:\Dileep\NIDHI_V02\degree8_sequences_only.txt"   # existing LFSR sequences
```

```
output_file = r"D:\Dileep\NIDHI_V02\correlation_output.txt"


# ------------------ Helper Functions ------------------


# ----- Read sequences for correlation -----
def read_sequences(filename):
    sequences = []
    with open(filename, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:  # ignore empty lines
                sequences.append(line)
    if not sequences:
        print(f"No sequences found in {filename}.")
        exit()
    return sequences


def detect_type(sequences):
    for seq in sequences:
        for d in seq:
            if d not in ['0', '1']:
                return "z4"
    return "binary"
```

```
# ---------- Binary Correlation ----------
def binary_correlation(seq1, seq2):
    agreement = sum(1 for a, b in zip(seq1, seq2) if a == b)
    disagreement = len(seq1) - agreement
    return agreement - disagreement


def circular_shift(seq, k):
    return seq[k:] + seq[:k]


# ---------- Z4 Helpers ----------
def simplify_complex_powers(powers):
    results = {"real": 0, "imaginary": 0}
    for p in powers:
        remainder = p % 4
        if remainder == 0:
            results["real"] += 1
        elif remainder == 1:
            results["imaginary"] += 1
        elif remainder == 2:
            results["real"] -= 1
        elif remainder == 3:
            results["imaginary"] -= 1
    r = results["real"]
    im = results["imaginary"]
```

```
    output = []
    if r != 0: output.append(f"{r}")
    if im != 0: output.append(f"{im}i")
    return " + ".join(output) if output else "0"

# ---------- Z4 Cross Correlation (Shift 0 Only) ----------
def z4_cross_correlation_shift0(seq1, seq2):
    diff = [(int(a) - int(b)) % 4 for a, b in zip(seq1, seq2)]
    result = simplify_complex_powers(diff)
    cleaned = result.replace('i', 'j').replace(' ', '').replace('+-', '-').replace('--', '+')
    magnitude = abs(complex(cleaned))
    return [(0, result, magnitude)], magnitude

# ---------- Auto Correlation ----------
def auto_correlation(sequences, mode):
    results = []
    for idx, seq in enumerate(sequences):
        results.append(f"\nSequence {idx + 1}: {seq}")
        if mode == "binary":
            n = len(seq)
            for shift in range(1, n):
                shifted = circular_shift(seq, shift)
                corr = binary_correlation(seq, shifted)
                results.append(f"Shift {shift} → {corr}")
```

```
        else:  # z4 auto
            powers = [int(d) for d in seq]
            result = simplify_complex_powers(powers)
            cleaned = result.replace('i', 'j').replace(' ', '').replace('+-', '-').replace('--', '+')
            magnitude = abs(complex(cleaned))
            results.append(f"Auto-correlation → {result} | magnitude={magnitude}")
            results.append(f"Peak auto-correlation magnitude: {magnitude}")
    return results


# ---------- Cross Correlation ----------
def cross_correlation(sequences, mode):
    results = []
    for (i, seq1), (j, seq2) in itertools.combinations(enumerate(sequences), 2):
        results.append(f"\nSequence {i + 1} vs Sequence {j + 1}")
        if mode == "binary":
            n = len(seq1)
            peak_corr = None
            for shift in range(n):
                shifted_seq2 = circular_shift(seq2, shift)
                corr = binary_correlation(seq1, shifted_seq2)
                if peak_corr is None or abs(corr) > abs(peak_corr):
                    peak_corr = corr
                results.append(f"Shift {shift} → {corr}")
            results.append(f"Peak cross-correlation magnitude: {abs(peak_corr)}")
```

```
        else:  # z4 modified cross, shift 0 only
            z4_results, peak_mag = z4_cross_correlation_shift0(seq1, seq2)
            for shift, res, mag in z4_results:
                results.append(f"Shift {shift} → {res} | magnitude={mag}")
            results.append(f"Peak cross-correlation magnitude: {peak_mag}")
    return results


# -------------------- MAIN --------------------
def main():
    # ---- Step 1: Read sequences ----
    sequences = read_sequences(input_file)
    print(f"Read {len(sequences)} sequences of length {len(sequences[0])}.")

    # ---- Step 2: Detect type ----
    mode = detect_type(sequences)
    print(f"Detected sequence type: {mode}")

    # ---- Step 3: Compute correlations ----
    output_lines = []
    output_lines.append(f"Detected sequence type: {mode}\n")
    output_lines.append("AUTO CORRELATION")
    output_lines.extend(auto_correlation(sequences, mode))
    output_lines.append("\nCROSS CORRELATION")
    output_lines.extend(cross_correlation(sequences, mode))
```

```
    # ---- Step 4: Write results ----
    with open(output_file, 'w', encoding='utf-8') as f:
        for line in output_lines:
            f.write(line + "\n")

    print(f"\nResults saved to {output_file}")

if __name__ == "__main__":
    main()
```

Step 3

opt filter

```
import re
import networkx as nx
import os

file_path = r"D:\Dileep\NIDHI_V02\correlation_output.txt"
out_path = r"D:\Dileep\NIDHI_V02\op.txt"

if not os.path.exists(file_path):
    print(f"ERROR: File not found at {file_path}")
    exit()
```

```
pairs = {}
auto_corr = {}
sequences = set()

print("Reading file and parsing data...")

with open(file_path, "r") as f:
    current_pair = None
    current_seq = None

    for line in f:
        line = line.strip()
        if not line: continue

        # 1. Capture Sequence IDs for Cross-Correlation
        # Matches: "Sequence 1 vs Sequence 2"
        pair_match = re.search(r"Sequence\s+(\d+)\s+vs\s+Sequence\s+(\d+)", line, re.I)
        if pair_match:
            current_pair = (int(pair_match.group(1)), int(pair_match.group(2)))
            sequences.update(current_pair)
            continue

        # 2. Capture Single Sequence IDs for Auto-Correlation
        # Matches: "Sequence 1:"
```

```
        single_match = re.search(r"^Sequence\s+(\d+):", line, re.I)
        if single_match:
            current_seq = int(single_match.group(1))
            sequences.add(current_seq)
            continue

        # 3. Capture Magnitude (Works for both Auto and Cross)
        mag_match = re.search(r"magnitude[:\s]+([0-9.]+)", line, re.I)
        if mag_match:
            val = float(mag_match.group(1))
            # If we just saw a pair, it's a cross-correlation value
            if current_pair:
                pairs[current_pair] = val
                current_pair = None
                # If we just saw a single sequence, it's an auto-correlation (PSL) value
            elif current_seq is not None:
                auto_corr[current_seq] = val
                current_seq = None

# --- Diagnostics ---
print(f"Data Loaded: {len(sequences)} sequences")
print(f"PSL values found: {len(auto_corr)}")
print(f"Cross-pairs found: {len(pairs)}")
```

```
if len(auto_corr) == 0:
    print("\nERROR: No PSL magnitudes found! Check if your file says 'magnitude: [value]'")
    exit()

# Show the user what was actually found so they can pick a good threshold
min_val = min(auto_corr.values())
max_val = max(auto_corr.values())
print(f"\nYour actual PSL values range from: {min_val:.2f} to {max_val:.2f}")

try:
    auto_threshold = float(input(f'Enter MAX PSL threshold (Try {int(min_val) + 1}): '))
    cross_threshold = float(input("Enter MAX CROSS threshold: "))
except ValueError:
    exit()

# Filter
valid_nodes = [s for s in sequences if auto_corr.get(s, 9999) <= auto_threshold]
print(f"Nodes passing PSL threshold: {len(valid_nodes)} / {len(sequences)}")

if not valid_nodes:
    print("Zero nodes passed. Threshold too low.")
    exit()

# Graph
```

```
G = nx.Graph()
G.add_nodes_from(valid_nodes)
for (a, b), mag in pairs.items():
    if a in valid_nodes and b in valid_nodes:
        if mag <= cross_threshold:
            G.add_edge(a, b)

print(f"Graph built with {G.number_of_edges()} edges.")

# Cliques
max_cliques = list(nx.find_cliques(G))
if not max_cliques:
    print("No cliques found.")
else:
    max_cliques.sort(key=len, reverse=True)
    with open(out_path, "w") as f:
        for c in max_cliques:
            c.sort()
            f.write("{" + ",".join(map(str, c)) + "}-" + str(len(c)) + "\n")
    print(f"Saved to: {out_path}")
```

Learning Outcomes

After studying these topics, the learner will be able to understand and explain the fundamental mathematical structures used in digital communication and signal processing systems. The learner will gain a clear understanding of Galois Field and its role in defining finite arithmetic systems used in computing and communication. They will be able to describe how operations in finite fields differ from real-number arithmetic and why these structures are essential in modern digital technologies.

The learner will also understand the working principles of the Linear Feedback Shift Register and how it is used to generate pseudorandom binary sequences. They will be able to explain how shift operations and feedback mechanisms produce deterministic yet complex sequences that are widely used in communication systems, cryptography, and testing.

In addition, the learner will develop knowledge of primitive polynomials and their importance in generating maximum-length sequences in LFSRs. They will be able to identify how these polynomials ensure proper cycle lengths and improved statistical properties.

Finally, the learner will understand how these concepts collectively support applications in error correction, encryption, and digital signal processing. They will be able to connect theoretical mathematical ideas with real-world engineering systems, improving both conceptual clarity and practical understanding of modern communication technologies

Conclusion

The study of finite algebraic structures and sequence generation techniques forms the foundation of many modern communication and computing systems. Concepts such as the Galois Field, Linear Feedback Shift Register, and primitive polynomials are not just abstract mathematical ideas but essential tools that enable reliable and efficient digital technology.

Galois Fields provide a structured way of performing arithmetic with a finite number of elements, which is particularly suitable for binary systems used in computers and digital circuits. Their properties make them highly useful in error detection, error correction, and secure data transmission. Many modern technologies, including data storage systems, wireless communication, and cryptographic algorithms, depend on finite field operations to function correctly and efficiently.

The Linear Feedback Shift Register plays a crucial role in generating pseudorandom sequences that are widely used in communication systems, cryptography, and digital signal processing. Its simplicity in hardware implementation combined with its ability to produce long, complex sequences makes it highly valuable in engineering applications. When designed properly, it provides sequences with good statistical properties and predictable structure.

Primitive polynomials further enhance the performance of LFSRs by ensuring maximum-length sequence generation and proper cycling through all nonzero states. This improves randomness, synchronization, and system reliability.

Overall, these concepts are deeply interconnected and form the mathematical backbone of modern digital communication systems. Understanding them provides insight into how information is securely transmitted, accurately processed, and efficiently managed in today's technology-driven world.